

Variational Autoencoders (VAEs)

Encoder, decoder, ELBO, reparameterization, and examples

Moin Mostakim

Outline

- 1 Motivation and setup
- 2 Model and ELBO
- 3 Reparameterization and training
- 4 Architectures
- 5 Working example
- 6 Practice problems

Generative modeling goal

- Observe data $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(N)}$ drawn from an unknown distribution $p_{\text{data}}(\mathbf{x})$.
- Want a tractable model $p_{\theta}(\mathbf{x})$ that assigns high probability to realistic samples and from which we can *sample* new data.
- A **latent variable** model posits a simple prior $p(\mathbf{z})$ (often $\mathcal{N}(\mathbf{0}, \mathbf{I})$) and a conditional $p_{\theta}(\mathbf{x} | \mathbf{z})$ implemented by a neural network (the **decoder**).

Challenge: marginal likelihood $p_{\theta}(\mathbf{x}) = \int p_{\theta}(\mathbf{x} | \mathbf{z})p(\mathbf{z}) d\mathbf{z}$ is usually intractable.

Variational inference idea

Introduce an approximate posterior $q_\phi(\mathbf{z} \mid \mathbf{x})$ (the **encoder**) and maximize a lower bound on $\log p_\theta(\mathbf{x})$ instead of the log-likelihood directly.

Key quantity: ELBO

For any $q_\phi(\mathbf{z} \mid \mathbf{x})$ with support where $p_\theta(\mathbf{x} \mid \mathbf{z}) > 0$,

$$\log p_\theta(\mathbf{x}) \geq \mathcal{L}(\mathbf{x}; \theta, \phi) := \mathbb{E}_{q_\phi(\mathbf{z} \mid \mathbf{x})}[\log p_\theta(\mathbf{x} \mid \mathbf{z})] - \text{KL}(q_\phi(\mathbf{z} \mid \mathbf{x}) \parallel p(\mathbf{z})).$$

Training maximizes $\mathcal{L}$ w.r.t. (θ, ϕ) (often written as minimizing $-\mathcal{L}$).

Latent variable model (decoder)

- Prior: $p(\mathbf{z}) = \mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I}_d)$, d is latent dimension.
- Likelihood (decoder): $p_\theta(\mathbf{x} | \mathbf{z})$. Common choices:
 - $\mathbf{x} \in \{0, 1\}^D$: Bernoulli logits from a network $f_\theta(\mathbf{z})$.
 - $\mathbf{x} \in \mathbb{R}^D$: Gaussian with fixed or predicted diagonal covariance.
- Example (Bernoulli): $p_\theta(\mathbf{x} | \mathbf{z}) = \prod_{j=1}^D \text{Ber}(x_j; \pi_j)$ where $\boldsymbol{\pi} = f_\theta(\mathbf{z})$.

Approximate posterior (encoder)

- Encoder $q_\phi(\mathbf{z} \mid \mathbf{x})$ approximates the true posterior $p_\theta(\mathbf{z} \mid \mathbf{x}) \propto p_\theta(\mathbf{x} \mid \mathbf{z})p(\mathbf{z})$.
- Standard VAE: **factorized Gaussian**

$$q_\phi(\mathbf{z} \mid \mathbf{x}) = \mathcal{N}(\mathbf{z}; \boldsymbol{\mu}_\phi(\mathbf{x}), \text{diag}(\boldsymbol{\sigma}_\phi^2(\mathbf{x}))),$$

where $\boldsymbol{\mu}_\phi, \log \boldsymbol{\sigma}_\phi^2$ are outputs of a neural network (two heads).

- Sampling: $\mathbf{z} = \boldsymbol{\mu}_\phi(\mathbf{x}) + \boldsymbol{\sigma}_\phi(\mathbf{x}) \odot \boldsymbol{\epsilon}, \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.

ELBO derivation (step 1)

Start from $\log p_{\theta}(\mathbf{x})$ and insert $q_{\phi}(\mathbf{z} \mid \mathbf{x})$:

$$\begin{aligned}\log p_{\theta}(\mathbf{x}) &= \log \int p_{\theta}(\mathbf{x} \mid \mathbf{z}) p(\mathbf{z}) d\mathbf{z} \\ &= \log \int q_{\phi}(\mathbf{z} \mid \mathbf{x}) \frac{p_{\theta}(\mathbf{x} \mid \mathbf{z}) p(\mathbf{z})}{q_{\phi}(\mathbf{z} \mid \mathbf{x})} d\mathbf{z} \\ &= \log \mathbb{E}_{q_{\phi}(\mathbf{z} \mid \mathbf{x})} \left[\frac{p_{\theta}(\mathbf{x} \mid \mathbf{z}) p(\mathbf{z})}{q_{\phi}(\mathbf{z} \mid \mathbf{x})} \right].\end{aligned}$$

Apply Jensen's inequality (log is concave): $\log \mathbb{E}[\cdot] \geq \mathbb{E}[\log(\cdot)]$.

ELBO derivation (step 2)

$$\begin{aligned}\log p_{\theta}(\mathbf{x}) &\geq \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\log \frac{p_{\theta}(\mathbf{x} | \mathbf{z}) p(\mathbf{z})}{q_{\phi}(\mathbf{z} | \mathbf{x})} \right] \\ &= \mathbb{E}_{q_{\phi}} [\log p_{\theta}(\mathbf{x} | \mathbf{z})] + \mathbb{E}_{q_{\phi}} \left[\log \frac{p(\mathbf{z})}{q_{\phi}(\mathbf{z} | \mathbf{x})} \right] \\ &= \underbrace{\mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} [\log p_{\theta}(\mathbf{x} | \mathbf{z})]}_{\text{expected log-likelihood (reconstruction)}} - \underbrace{\text{KL}(q_{\phi}(\mathbf{z} | \mathbf{x}) \| p(\mathbf{z}))}_{\text{regularizer}}.\end{aligned}$$

The gap $\log p_{\theta}(\mathbf{x}) - \mathcal{L}(\mathbf{x})$ equals $\text{KL}(q_{\phi} \| p_{\theta}(\mathbf{z} | \mathbf{x})) \geq 0$.

KL term for Gaussian encoder and standard prior

Let $q = \mathcal{N}(\boldsymbol{\mu}, \text{diag}(\boldsymbol{\sigma}^2))$ and $p = \mathcal{N}(\mathbf{0}, \mathbf{I})$. Then (per dimension, summed):

$$\text{KL}(q \parallel p) = \frac{1}{2} \sum_{i=1}^d \left(\sigma_i^2 + \mu_i^2 - 1 - \log \sigma_i^2 \right).$$

In code, predict $\log \sigma_i^2$ for numerical stability and substitute $\sigma_i^2 = \exp(\log \sigma_i^2)$.

Reparameterization trick

Goal: unbiased gradient of $\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}[g(\mathbf{z})]$ w.r.t. ϕ .

- Naive sampling $\mathbf{z} \sim q_\phi(\cdot | \mathbf{x})$ blocks gradients through ϕ (discrete “path”).
- Write $\mathbf{z} = h_\phi(\mathbf{x}, \epsilon)$ with $\epsilon \sim p(\epsilon)$ independent of ϕ :

$$\mathbf{z} = \boldsymbol{\mu}_\phi(\mathbf{x}) + \boldsymbol{\sigma}_\phi(\mathbf{x}) \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}).$$

- Then $\nabla_\phi \mathbb{E}_\epsilon[\log p_\theta(\mathbf{x} | h_\phi(\mathbf{x}, \epsilon))] = \mathbb{E}_\epsilon[\nabla_\phi \log p_\theta(\mathbf{x} | h_\phi(\mathbf{x}, \epsilon))]$.

Single-datapoint objective (minimization form)

For one $\mathbf{x}$, Monte Carlo estimate with L samples (often $L = 1$):

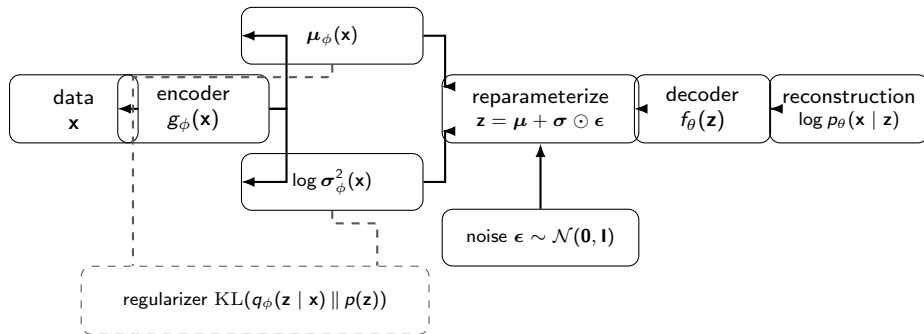
$$\mathcal{L}(\mathbf{x}) \approx \frac{1}{L} \sum_{\ell=1}^L \log p_{\theta}(\mathbf{x} \mid \mathbf{z}^{(\ell)}) - \text{KL}(q_{\phi}(\mathbf{z} \mid \mathbf{x}) \parallel p(\mathbf{z})), \quad \mathbf{z}^{(\ell)} = h_{\phi}(\mathbf{x}, \epsilon^{(\ell)}).$$

Loss to minimize: $J(\mathbf{x}) = -\mathcal{L}(\mathbf{x}) = \text{reconstruction term} + (\text{weighted}) \text{ KL}.$

Algorithm: VAE training (minibatch)

- 1 Sample minibatch $\{\mathbf{x}^{(n)}\}_{n=1}^B$.
- 2 For each $\mathbf{x}^{(n)}$: compute $\boldsymbol{\mu}_\phi, \log \boldsymbol{\sigma}_\phi^2$; sample $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$; $\mathbf{z} = h_\phi(\mathbf{x}^{(n)}, \boldsymbol{\epsilon})$.
- 3 Decoder: evaluate $\log p_\theta(\mathbf{x}^{(n)} | \mathbf{z})$ (e.g. sum of Bernoulli log-probs).
- 4 Add analytic $\text{KL}(q_\phi \| p)$ for Gaussians.
- 5 Average over batch; backprop; SGD step on (θ, ϕ) .

VAE architecture (segmented pipeline)



- **Encoder segment g_ϕ :** maps x to moments of $q_\phi(z | x)$.
- **Stochastic segment:** same z feeds the decoder; gradients flow through μ, σ via ϵ .
- **Decoder segment f_θ :** maps z to likelihood parameters (e.g. Bernoulli logits or Gaussian mean).

Encoder and decoder (detail)

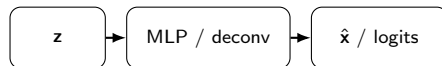
Encoder (inference)

- Input $\mathbf{x}$ (e.g. flattened 28×28 image).
- CNN or MLP $\rightarrow$ hidden $\mathbf{h}$.
- Two heads: $\mu_{\phi}(\mathbf{x}), \log \sigma_{\phi}^2(\mathbf{x}) \in \mathbb{R}^d$.



Decoder (generative)

- Input $\mathbf{z} \in \mathbb{R}^d$; MLP / deconv $\rightarrow p_{\theta}(\mathbf{x} | \mathbf{z})$.
- Bernoulli: logits $\hat{\mathbf{x}}$; Gaussian: $\mu_{\theta}(\mathbf{z})$ (and optional σ_{θ}).



Toy setup: $D = 2$, $d = 1$ (conceptual)

Suppose $\mathbf{x} \in \mathbb{R}^2$, latent $z \in \mathbb{R}$, prior $p(z) = \mathcal{N}(0, 1)$. Encoder could be a tiny network; decoder predicts $\hat{\mathbf{x}} \in (0, 1)^2$ after sigmoid if we treat $\mathbf{x}$ as probabilities / preprocessed to $[0, 1]$.

Forward pass numbers (illustrative):

- $\mu_\phi(\mathbf{x}) = 0.3$, $\log \sigma_\phi^2 = -0.5 \Rightarrow \sigma \approx 0.78$.
- Draw $\epsilon = -0.2 \Rightarrow z = 0.3 + 0.78(-0.2) \approx 0.14$.
- Decoder gives $\hat{x}_1 = 0.7$, $\hat{x}_2 = 0.4$; data $x_1 = 1$, $x_2 = 0$.
- Bernoulli reconstruction (sum): $\log 0.7 + \log(1 - 0.4) \approx -0.36 + (-0.51) = -0.87$.
- KL (scalar): $\frac{1}{2}(\sigma^2 + \mu^2 - 1 - \log \sigma^2) \approx 0.5(0.61 + 0.09 - 1 + 0.5) \approx 0.10$.
- ELBO $\approx -0.87 - 0.10 = -0.97$; minimize negative ELBO.

Pseudocode: forward pass (MNIST-sized MLP)

Algorithm 1: VAE forward map (Bernoulli decoder)

Require: $\mathbf{x} \in [0, 1]^D$; parameters θ, ϕ ; latent dimension d

① $\mathbf{h} \leftarrow \text{ReLU}\left(W_{\phi}^{(1)}\mathbf{x} + b_{\phi}^{(1)}\right)$

encoder trunk (MLP/CNN)

② $\boldsymbol{\mu}_{\phi} \leftarrow W_{\phi}^{(\mu)}\mathbf{h} + b_{\phi}^{(\mu)} \in \mathbb{R}^d$

③ $\mathbf{v}_{\phi} \leftarrow W_{\phi}^{(\nu)}\mathbf{h} + b_{\phi}^{(\nu)} \in \mathbb{R}^d$

$$\mathbf{v}_{\phi} = \log \sigma_{\phi}^2$$

④ draw $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_d)$

independent of ϕ

⑤ $\mathbf{z} \leftarrow \boldsymbol{\mu}_{\phi} + \exp(\frac{1}{2}\mathbf{v}_{\phi}) \odot \boldsymbol{\epsilon}$

reparameterization

⑥ $\mathbf{u} \leftarrow W_{\theta}^{(2)} \text{ReLU}\left(W_{\theta}^{(1)}\mathbf{z} + b_{\theta}^{(1)}\right) + b_{\theta}^{(2)}$

decoder MLP

⑦ $\hat{\mathbf{x}} \leftarrow \sigma(\mathbf{u})$

Bernoulli probabilities

return $\hat{\mathbf{x}}, \boldsymbol{\mu}_{\phi}, \mathbf{v}_{\phi}$

Pseudocode: ELBO loss (one Monte Carlo sample)

Algorithm 2: loss for one $\mathbf{x}$ (minimize)

Require: $\mathbf{x}; \hat{\mathbf{x}}, \boldsymbol{\mu}_\phi, \mathbf{v}_\phi$ from Alg. 1, with $\mathbf{v}_\phi = \log \boldsymbol{\sigma}_\phi^2$

$$\textcircled{1} \mathcal{L}_{\text{rec}} \leftarrow - \sum_{j=1}^D (x_j \log \hat{x}_j + (1 - x_j) \log(1 - \hat{x}_j))$$

Bernoulli NLL

$$\textcircled{2} \mathcal{L}_{\text{KL}} \leftarrow \frac{1}{2} \sum_{i=1}^d (e^{v_i} + \mu_i^2 - 1 - v_i)$$

$v_i = (\mathbf{v}_\phi)_i$

$$\textcircled{3} J \leftarrow \mathcal{L}_{\text{rec}} + \mathcal{L}_{\text{KL}}$$

optional β on $\mathcal{L}_{\text{KL}}$ (β -VAE)

return J

Minibatch training: average J over $\{\mathbf{x}^{(n)}\}$, stochastic gradient on (θ, ϕ) (e.g. Adam).

- **Reconstruction:** encode $\mathbf{x}$, sample or use $\boldsymbol{\mu}$ for $\mathbf{z}$, decode to $\hat{\mathbf{x}}$.
- **Generation:** sample $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, run decoder $\rightarrow$ synthetic $\hat{\mathbf{x}}$.
- **Interpolation:** linearly interpolate $\mathbf{z}$ between two encoded means, decode intermediate points.

Practice problems (1–2)

- 1 Starting from $\mathcal{L} = \mathbb{E}_q[\log p_\theta(\mathbf{x} | \mathbf{z})] - \text{KL}(q \| p)$, show that maximizing $\mathcal{L}$ w.r.t. q (with fixed θ) is equivalent to minimizing $\text{KL}(q_\phi(\mathbf{z} | \mathbf{x}) \| p_\theta(\mathbf{z} | \mathbf{x}))$.
- 2 Derive the closed-form $\text{KL}(\mathcal{N}(\boldsymbol{\mu}, \text{diag}(\boldsymbol{\sigma}^2)) \| \mathcal{N}(\mathbf{0}, \mathbf{I}))$ for arbitrary dimension d .

Practice problems (3–5)

- 3 Suppose the decoder is Gaussian $p_{\theta}(\mathbf{x} \mid \mathbf{z}) = \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_{\theta}(\mathbf{z}), \sigma^2 \mathbf{I})$ with fixed σ^2 . Write the reconstruction term $\mathbb{E}_q[\log p_{\theta}(\mathbf{x} \mid \mathbf{z})]$ as an expectation of a squared error plus constants.
- 4 Explain why the naive Monte Carlo gradient $\nabla_{\phi} \mathbb{E}_{\mathbf{z} \sim q_{\phi}}[f(\mathbf{z})]$ using “score function” / REINFORCE can have higher variance than the reparameterization gradient for smooth f .
- 5 If $\text{KL}(q_{\phi}(\mathbf{z} \mid \mathbf{x}) \| p(\mathbf{z})) \approx 0$ for all training points, what might that indicate about q_{ϕ} and the latent codes? What failure mode of generation could follow?

Practice problems (6–7) — short implementations

- 6 Implement a 1D VAE on synthetic data drawn from a mixture of two Gaussians in $\mathbb{R}^2$; plot encoded means and generated samples.
- 7 Replace the Gaussian encoder with a **discrete** latent (e.g. categorical with Gumbel-Softmax). Which part of the standard VAE derivation breaks, and what trick restores differentiability?

- Kingma & Welling, “Auto-Encoding Variational Bayes” (ICLR 2014).
- Rezende, Mohamed, Wierstra, “Stochastic Backpropagation and Approximate Inference in Deep Generative Models” (ICML 2014).